The vulnerability here revolves around the fact that browsers automatically send cookies when a page is requested (using a `GET` request) or a form is `POST`ed. There's no difference between a legitimate `POST` of the form in your banking app and the attacker's malicious `POST`. Unfortunately, this behavior is baked into the web; it's what allows you to navigate websites seamlessly after initially logging in.

TIP Browsers have additional protections to prevent cookies being sent in this situation, called SameSite cookies. By default, most browsers use `SameSite=Lax`, which prevents this vulnerable behavior. You can read about SameSite cookies and how to work with them in ASP.NET Core at http://mng.bz/4DDj.

A common solution to this CSRF attack is the *synchronizer token* pattern, which uses user-specific, unique antiforgery tokens to enforce a difference between a legitimate `POST` and a forged `POST` from an attacker. One token is stored in a cookie, and another is added to the form you wish to protect. Your app generates the tokens at runtime based on the current logged-in user, so there's no way for an attacker to create one for their forged form.

TIP The "Cross-Site Request Forgery Prevention Cheat Sheet" article on the OWASP site (http://mng.bz/5jRa) has a thorough discussion of the CSRF vulnerability, including the synchronizer token pattern.

When the `Balance` Razor Page receives a form `POST`, it compares the value in the form with the value in the cookie. If either value is missing or the values don't match, the request is rejected. If an attacker creates a `POST`, the browser posts the cookie token as usual, but there won't be a token in the form itself or the token won't be valid. The Razor Page rejects the request, protecting from the CSRF attack, as in figure 29.5.